Using Flutter packages

Projects: `code/week07/examples` and `maps_examples`

Key ideas

- **Package**: reusable libraries + metadata. Pure Dart (no widgets) or Flutter (widgets).
- **Plugin**: a package that calls platform code (location, maps). The same Dart API does not mean every platform is supported.
- Adding a dependency ≠ importing a library: two steps.

`intl`	number formatting with a locale
`http`	requests and responses (week 8)
`flutter_lints`	analyzer rules (dev only)
`area`	our local package
`google_maps_flutter`	map view, camera, markers
`location`	device location and permission

Choosing a package on pub.dev

Read: README, **supported platforms**, SDK requirement, publisher, license, release history, examples, issues. Does it need native setup, an API key, billing or runtime permissions? Popularity alone is not suitability.

Add and import

```
flutter pub add http intl   # updates pubspec
flutter pub add --dev flutter_lints
flutter pub get             # resolve + download
flutter pub outdated        # what could update
```

```
dependencies:        # spaces, not tabs
  flutter:
    sdk: flutter
  area:
    path: packages/area
  intl: ^0.20.3
  http: ^1.6.0
```

```
import 'package:http/http.dart' as http;
import 'package:intl/intl.dart';
```

A dependency is a sibling of `flutter`, not a child of `sdk`. `pub add` may pick an older version if constraints require it; `pub get` does not upgrade everything.

Versions and the lockfile

Constraint	Allows
`^1.6.0`	$\geq$ 1.6.0 and $<$ 2.0.0
`^0.13.5`	$\geq$ 0.13.5 and $<$ 0.14.0
`^0.0.3`	$\geq$ 0.0.3 and $<$ 0.0.4

- `pubspec.yaml`: ranges. `pubspec.lock`: exact versions, transitive ones included. Commit an app's lockfile.
- A package update does not reach apps by itself: their constraints and lockfiles decide.

intl: format numbers

```
NumberFormat('#,##0.00', 'en_US')
    .format(1234.567);     // 1,234.57
NumberFormat('#,##0.00', 'de_DE')
    .format(1234.567);     // 1.234,57
NumberFormat('0.####', 'en_US')
    .format(12.34567);     // 12.3457
```

0: a digit is required; #: optional. Formatting changes the display, not the stored number.

Dev dependencies and lints

```
# analysis_options.yaml
include: package:flutter_lints/flutter.yaml
```

- `flutter analyze`: a lint is advice, not a compile error or a crash.
- Dev dependencies (tests, lints) are not given to the package's users: never import them in production code.
- App size depends on code actually used (tree shaking), not on the number of dependencies.

Your own package: area

```
dart create -t package area   # pure Dart
dart pub add intl
dart pub add --dev test
# widgets? flutter create --template=package name
```

```
packages/area/
  pubspec.yaml  README.md  CHANGELOG.md
  lib/area.dart            # public entry
  lib/src/shapes.dart
  lib/src/formatting.dart
  test/area_test.dart
```

```
// lib/area.dart
library;
export 'src/shapes.dart';
export 'src/formatting.dart';

// lib/src/shapes.dart
double rectangleArea(double w, double h) {
  _checkDimensions(w, h); // finite, > 0
  return w * h;
}
```

- Users import `package:area/area.dart`. `lib/src` is a convention, not an access barrier.
- Calculations return `double`; formatting returns a `String`: keep them apart.
- Validate in the package: it protects every caller.

Use the local package

```
import 'package:area/area.dart';
final value = triangle
    ? triangleArea(width, height)
    : rectangleArea(width, height);
setState(() => _result =
  'Area: ${formatArea(value)} square units');
```

`intl` of `area` becomes a **transitive** dependency of the app. Width 4, height 3: rectangle 12, triangle 6. Invalid text is not zero: validate it.

Copyrighted content Dr Mohamad AOUDE

export versus part

`export`	each file is its own library, own imports
`part` / `part of`	splits *one* library: imports and private names shared

```
// area.dart
part 'src/rectangle.dart';
// src/rectangle.dart
part of '../area.dart';
```

A part belongs to a library, not to the package. Prefer `export` for handwritten code; generators use parts.

Share and publish

```
area:
  git:
    url: https://example.org/team/repo.git
    ref: <commit sha>    # a branch can move
    path: packages/area
```

```
dart analyze
dart test
dart pub publish --dry-run   # checks only
```

- Before publishing: name, description, version, README, CHANGELOG, tests, your own LICENSE.
- `publish_to: 'none'` prevents publishing.
- `dart pub publish` makes it public: not in this course.

Google Maps: setup first

1. Add `google_maps_flutter`; generate the platform runner.
2. Google Cloud project, billing, Maps API per platform.
3. Restricted API keys in the native or web runner.
4. Run a fixed map before adding location.

Android	`com.google.android.geo.API_KEY` in the manifest `application`; key restricted to app ID + certificate
iOS	`GMSServices.provideAPIKey`; `NSLocationWhenInUseUsageDescription`
Web	Maps JavaScript API script before Flutter starts; restrict to referrers; HTTPS or localhost

Follow the README of the resolved plugin version; pricing changes: check the official page.

A map, a camera, markers

```
GoogleMap(
  initialCameraPosition: const CameraPosition(
    target: LatLng(33.8938, 35.5018), // Beirut
    zoom: 13),
  onMapCreated: (c) => _controller = c,
  markers: {
    const Marker(
      markerId: MarkerId('sample-a'),
      position: LatLng(33.895, 35.503),
      infoWindow: InfoWindow(
        title: 'Sample restaurant A',
        snippet: 'Teaching data')),
  },
)
await _controller?.animateCamera(
  CameraUpdate.newLatLngZoom(point, 15));
```

- Give the map a bounded height (`Expanded` in a `Column`).
- Move an existing map with `animateCamera`/`moveCamera`, not by changing `initialCameraPosition`.
- `markers` is a `Set<Marker>`; each needs a stable `MarkerId`. Build a *new* set in `setState`.

Foreground location

```
if (!await gw.serviceEnabled() &&
    !await gw.requestService()) {
  throw StateError('Enable location services.');
}
var p = await gw.hasPermission();
if (p == PermissionStatus.denied) {
  p = await gw.requestPermission();
}
if (p != PermissionStatus.granted &&
    p != PermissionStatus.grantedLimited) {
  throw StateError('Permission unavailable.');
}
return gw.coordinates()
    .timeout(const Duration(seconds: 15));
```

- Checking permission does not request it. Permanent denial: send the user to settings.
- Ask only when the user taps the button; disable it while busy; check `mounted` after each `await`.
- A gateway interface lets tests use a fake device.
- Never present the default point as the user's position.

A key in Dart code or in `--dart-define` can be extracted from the app. Web-service keys (Places) belong on a backend; enable and restrict each Google service separately.

Common errors

Import compiles, plugin missing on device	generate and configure the runner; restart after adding a plugin
Generated package tests deleted	replace them with tests of the public API
Invalid input becomes 0	validate before calculating
Map key used for Places	enable and restrict each service
`Future<LatLng>` field never set	request on a button tap

Copyrighted content Dr Mohamad AOUDE